

Information has an arrival time

Daily research note | 21 September 2026 | Experiment 001

Sidiq review status: Not yet reviewed.

Today's question

Can a historical observation be used in a decision simply because its observation date is in the past? The hypothesis is no: publication delays and later revisions can introduce future information into a backtest. Today's component selects only the latest observation version actually available at the decision time.

Why this matters

A backtest simulates decisions using historical information. If it gives yesterday's simulated investor a number published tomorrow, the experiment has an unfair advantage. More sophisticated modelling cannot repair this error. Point-in-time selection is shared infrastructure for earnings signals, economic releases, options quotes and trade events.

Beginner explanation

Imagine a company measures earnings on January 1. It publishes 100 on January 3, then corrects that number to 60 on January 5. A database downloaded later may show only 60 next to January 1. Joining that database on January 1 would make the correction appear known four days early. Before January 3, the answer is missing. On January 3-4, the known value is 100. On January 5, the revision becomes usable.

Definitions

Observation time: when the measured economic event belongs. **Availability time:** the earliest instant a specific version could enter the research process. **Decision time:** when a simulated decision is made. **Revision:** a later version of an existing observation. **Look-ahead bias:** using information unavailable at the historical decision. **Latency:** additional processing or transmission delay. **Staleness:** age measured from the observation time. **Feature:** an input to a model or rule. **Backtest:** a historical simulation, not proof of a live edge.

Worked synthetic example

All dates below are January 2026 at 00:00 UTC. The numbers are invented solely to test software logic.

Decision day	Incorrect final-vintage join	Correct available value
1	60	Missing
2	60	Missing
3	60	100
4	60	100
5	60	60
6	60	60

The incorrect join differs at four of six decision times. This is a deliberately constructed counterexample, not an estimate of real-world error frequency and not a trading-performance result.

Formal model

Represent a record r by asset a , feature f , observation time $o(r)$, availability time $v(r)$ and scalar value $x(r)$. Require timezone-aware timestamps, finite values and $v(r) \geq o(r)$. Let t be decision time, $L \geq 0$ processing latency, and $A \geq 0$ an optional maximum observation age.

Eligibility: $E(t) = \{r : v(r) + L \leq t \text{ and, if } A \text{ is set, } t - o(r) \leq A\}$.

For each (a, f) , select the record in $E(t)$ with greatest pair $(o(r), v(r))$ in lexicographic order: choose the most recent observation first, then its most recent available revision. If $E(t)$ contains no record for a key, omit that key. Never fill it with a future value. Duplicate identical version keys are rejected as an ingestion ambiguity, even if their numeric values agree.

Non-anticipation property: adding records whose availability is after t must not change the snapshot at t , provided the appended records are valid and do not introduce duplicate version keys.

Lexicographic ordering matters. If an old January 1 observation is revised after a January 2 observation becomes available, that old revision must not replace the newer observation in a latest-observation snapshot. A full historical-vintage query would be a different API and is not implemented today.

Implementation

Observation is an immutable record. snapshot scans records once, validates duplicate keys, filters eligibility, and keeps the greatest eligible pair for each asset-feature key. Runtime is $O(n)$; storage is $O(n + k)$, where n is the number of records and k the number of keys, because duplicate detection retains version identifiers. The implementation uses the Python standard library and requires no numerical runtime dependency.

At exact publication time, a record is allowed by default. This is an explicit convention, not a claim of zero real-world delay. Callers should supply measured latency or an availability time incorporating vendor receipt. Maximum age is based on observation time; revising an old number does not make its economic content young again.

Tests and checks performed

21 unit tests passed in the initial core run. They cover future publication, revision timing, observation priority, input-order invariance, latency boundaries, staleness, separate assets/features, missing data, finite values, blank identifiers, invalid chronology, timezone requirements, negative durations, duplicate versions, equivalent timezones, future-data invariance and generator inputs. Ruff checks and formatting passed. Strict mypy passed for all 11 source files. The six-decision experiment ran successfully and wrote its JSON result. Additional reporting and research-artifact checks are recorded in `reports/milestone/initial-validation.md`.

Result and unsuccessful approach

The deliberately invalid final-vintage join fails at four decision times. The availability-aware selector agrees with the complete hand-worked release schedule. This establishes the intended behaviour for this fixture and the tested edge cases. It does not establish that any external dataset supplies trustworthy timestamps.

Limitations and possible error sources

Vendor receipt times, publication corrections, exchange calendars, corporate actions and changing universes are not solved by this component. A wrong availability timestamp still produces a wrong result. Fundamental fiscal period ends may need separate economic-date semantics. Missing observations are omitted, so downstream code must report missingness rather than silently reducing the universe. Duplicate versions fail the entire query, including future duplicates: this is deliberate data-quality validation, not a value-selection feature.

The package currently implements a reference scan rather than a high-throughput indexed history store. There is no price ingestion, signal model, portfolio return, transaction-cost estimate or live trading capability yet. Research claims must remain narrower than the long-term roadmap.

What to explain in an interview

Explain why observation date and publication date differ; use the 100-to-60 example without referring to code. Describe why chronological splitting alone does not fix revised-data leakage. Explain the two-stage selection rule, its $O(n)$ cost, and how measured latency affects the equality boundary. State what this component does not solve, especially survivorship bias and overlapping forward-return labels.

Next logical task

Q002: implement a chronological validation splitter that purges training observations whose target intervals overlap the evaluation period. The research question is whether a training label containing future test-period returns can leak information even when all features were constructed correctly. Write touching-boundary tests first.

Check your understanding

1. Which value is available on January 2, and why?
2. Why must an old observation's late revision not displace a newer observation?
3. If publication is at 10:00 and latency is one minute, is a 10:00 decision allowed to use the record?
4. Does passing these tests prove that a downloaded fundamentals dataset is point-in-time safe?
5. Why can a chronological train/test split still leak information?

Suggested answers

1. No value is available: the first publication is January 3. An earlier observation date does not imply earlier knowledge.
2. The requested snapshot represents the newest economic observation. Revision time resolves versions of the same observation; it does not determine which economic period is newest.
3. No. The earliest eligible decision is 10:01 under the inclusive boundary convention.
4. No. The tests check selection logic. Source timestamps, revisions, universe history and ingestion quality still need auditing.
5. A training target can include returns realised during the evaluation window, or a historical feature may contain a later revision. Label-interval purging and availability-aware features address different leakage paths.